COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 010 / Data Structures and Algorithms	**SECTION:**	CPE21S3
DATE PERFORMED:	September 17, 2026	**DATE SUBMITTED:**	September 17, 2026
NAME:	Reyes, Fenzie Aaron A.	**INSTRUCTOR:**	Engr. Jimlord Quejado

ACTIVITY NO. 9.1 **TREE ADT**

1. PROCEDURE OUTPUT

What procedures did you perform?

TASK 1 AND TASK 3 COMBINED: Creating the Code

main.cpp

Tree.h main.cpp

main.cpp > ...

```cpp
#include <iostream>
#include "Tree.h"

int main() {
    // Task 1: Building tree nodes using your BST class style
    BinarySearchTree<char>* tree = new BinarySearchTree<char>('A');

    // Create child nodes
    BinarySearchTree<char>* nodeB = new BinarySearchTree<char>('B');
    BinarySearchTree<char>* nodeC = new BinarySearchTree<char>('C');
    BinarySearchTree<char>* nodeD = new BinarySearchTree<char>('D');
    BinarySearchTree<char>* nodeE = new BinarySearchTree<char>('E');
    BinarySearchTree<char>* nodeF = new BinarySearchTree<char>('F');
    BinarySearchTree<char>* nodeG = new BinarySearchTree<char>('G');
    BinarySearchTree<char>* nodeH = new BinarySearchTree<char>('H');
    BinarySearchTree<char>* nodeI = new BinarySearchTree<char>('I');
    BinarySearchTree<char>* nodeJ = new BinarySearchTree<char>('J');
    BinarySearchTree<char>* nodeK = new BinarySearchTree<char>('K');
    BinarySearchTree<char>* nodeL = new BinarySearchTree<char>('L');
    BinarySearchTree<char>* nodeM = new BinarySearchTree<char>('M');
    BinarySearchTree<char>* nodeN = new BinarySearchTree<char>('N');
    BinarySearchTree<char>* nodeP = new BinarySearchTree<char>('P');
    BinarySearchTree<char>* nodeQ = new BinarySearchTree<char>('Q');
```

```
24
25          tree->addChild(nodeB);
26          tree->addChild(nodeC);
27          tree->addChild(nodeD);
28          tree->addChild(nodeE);
29          tree->addChild(nodeF);
30          tree->addChild(nodeG);
31
32          nodeD->addChild(nodeH);
33          nodeE->addChild(nodeI);
34          nodeE->addChild(nodeJ);
35          nodeF->addChild(nodeK);
36          nodeF->addChild(nodeL);
37          nodeF->addChild(nodeM);
38          nodeG->addChild(nodeN);
39          nodeJ->addChild(nodeP);
40          nodeJ->addChild(nodeQ);
```

```
42      // Task 3.2: Output Traversals
43      std::cout << "Pre Order Output: ";
44      tree->preOrder();
45      std::cout << std::endl;
46
47      std::cout << "In Order Output: ";
48      tree->inOrder();
49      std::cout << std::endl;
50
51      std::cout << "Post Order Output: ";
52      tree->postOrder();
53      std::cout << std::endl << std::endl;
54
55      // Task 3.3: findData function test
56      tree->findData("Pre-Order", 'P', tree);
57      std::cout << std::endl;
58
59      // Task 3.4: Add node 'O' as child of 'G' and search
60      BinarySearchTree<char>* nodeO = new BinarySearchTree<char>('O');
61      nodeG->addChild(nodeO);
62
63      tree->findData("Pre-Order", 'O', tree);
64
65      return 0;
66  }
```

Tree.h

Tree.h main.cpp

Tree.h > findData(std::string, T, BinarySearchTree<T>*)

```cpp
#ifndef Tree_H
#define Tree_H

#include <iostream>
#include <string>

template <typename T>
class BinarySearchTree {
private:
    T key;
    BinarySearchTree<T>* firstChild;   // Replaces left pointer for general trees
    BinarySearchTree<T>* nextSibling;  // Replaces right pointer for general trees

    bool searchTree(BinarySearchTree<T>* node, T target) {
        if (node == nullptr) return false;
        if (node->key == target) return true;
        if (searchTree(node->firstChild, target)) return true;
        return searchTree(node->nextSibling, target);
    }

public:
    BinarySearchTree(T value);
    void addChild(BinarySearchTree<T>* child);
    BinarySearchTree<T>* findNode(T value);

    // Traversal algorithms
    void preOrder();
    void inOrder();
    void postOrder();

    // Search algorithm
    void findData(std::string choice, T target, BinarySearchTree<T>* root);
};
```

```cpp
// Constructor implementation
template <typename T>
BinarySearchTree<T>::BinarySearchTree(T value) {
    key = value;
    firstChild = nullptr;
    nextSibling = nullptr;
}

// Add a child node
template <typename T>
void BinarySearchTree<T>::addChild(BinarySearchTree<T>* child) {
    if (firstChild == nullptr) {
        firstChild = child;
    } else {
        BinarySearchTree<T>* temp = firstChild;
        while (temp->nextSibling != nullptr) {
            temp = temp->nextSibling;
        }
        temp->nextSibling = child;
    }
}
```

```cpp
// Find node pointer by key
template <typename T>
BinarySearchTree<T>* BinarySearchTree<T>::findNode(T value) {
    if (key == value) return this;
    BinarySearchTree<T>* result = nullptr;
    if (firstChild != nullptr) result = firstChild->findNode(value);
    if (result == nullptr && nextSibling != nullptr) result = nextSibling->findNode(value);
    return result;
}
```

```cpp
// Pre-Order Traversal (Root -> Children)
template <typename T>
void BinarySearchTree<T>::preOrder() {
    std::cout << key << "    ";
    if (firstChild != nullptr) {
        firstChild->preOrder();
    }
    if (nextSibling != nullptr) {
        nextSibling->preOrder();
    }
}
```

```cpp
// In-Order Traversal
template <typename T>
void BinarySearchTree<T>::inOrder() {
    if (firstChild != nullptr) {
        firstChild->inOrder();
    }
    std::cout << key << "    ";
    if (nextSibling != nullptr) {
        nextSibling->inOrder();
    }
}
```

```cpp
// Post-Order Traversal
template <typename T>
void BinarySearchTree<T>::postOrder() {
    if (firstChild != nullptr) {
        firstChild->postOrder();
    }
    std::cout << key << "    ";
    if (nextSibling != nullptr) {
        nextSibling->postOrder();
    }
}
```

```cpp
// findData method
template <typename T>
void BinarySearchTree<T>::findData(std::string choice, T target, BinarySearchTree<T>* root) {
    std::cout << "Searching for '" << target << "' using [" << choice << "] traversal...\n";
    std::cout << "Traversal path: ";
    if (choice == "pre-order") {
        root->preOrder();
    } else if (choice == "in-order") {
        root->inOrder();
    } else if (choice == "post-order") {
        root->postOrder();
    }
    std::cout << std::endl;

    if (searchTree(root, target)) {
        std::cout << "\"" << target << " was found!\"\n";
    }
}

#endif
```

TASK 2: Completing the Table

NODE	HEIGHT	DEPTH
A	3	0
B	O	1
C	0	1
D	1	1
E	2	1
F	1	1
G	1	1
H	O	2
I	0	2
J	1	2
K	0	2
L	0	2
M	0	2
N	0	2
P	0	3
Q	0	3

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Copy and paste the questions from the lab and include your answers after.

Code + Console Screenshot + Tree Diagram

```
#ifndef BST_H
#define BST_H


template <typename T>
class BinarySearchTree{
    private:
        T key;
        BinarySearchTree<T>* left;
        BinarySearchTree<T>* right;

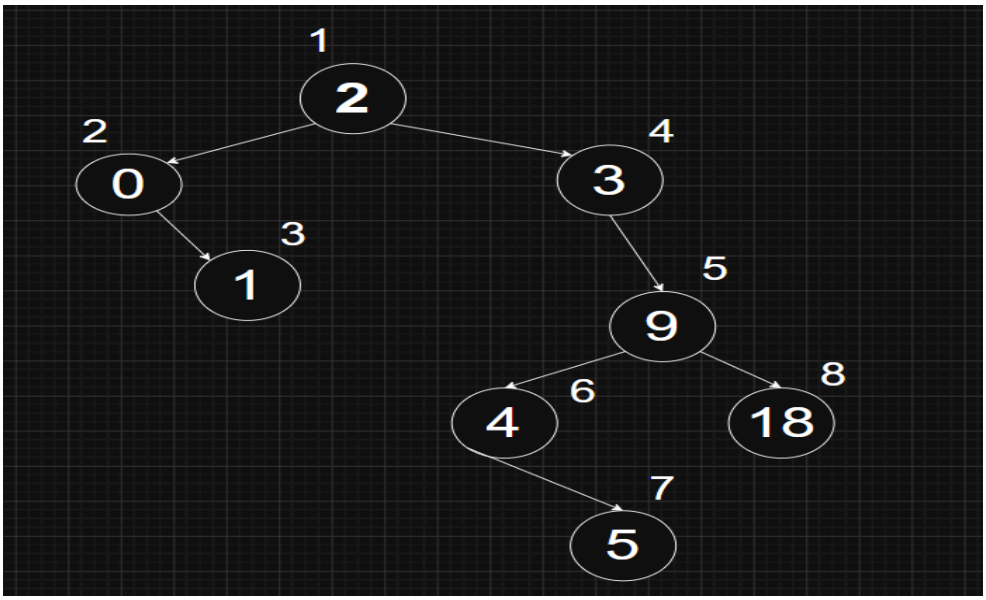
    public:
        // constructor
        BinarySearchTree(T value);
        // insertion algorithm:
        void insert(T value);

        // traversal
        void preOrder();
        void inOrder();
        void postOrder();
```

```
template<typename T>
void BinarySearchTree<T>::preOrder(){
    std::cout<< key << "    ";
    if(left != nullptr){
        left->preOrder();
    }
    if(right != nullptr){
        right->preOrder();
    }
}
```

```
    std::cout<<"Pre Order Output: ";
    tree->preOrder();
    std::cout<<std::endl;
```

```
Pre Order Output: 2    0    1    3    9    4    5    18
```



Observation	The pre-order traversal was successfully implemented using the Binary Search Tree. The function first displays the current node, then recursively visits its left child followed by its right child. Based on the given values inserted into the tree, the output follows the root-left-right sequence. This shows how pre-order traversal visits the parent node before its child nodes.

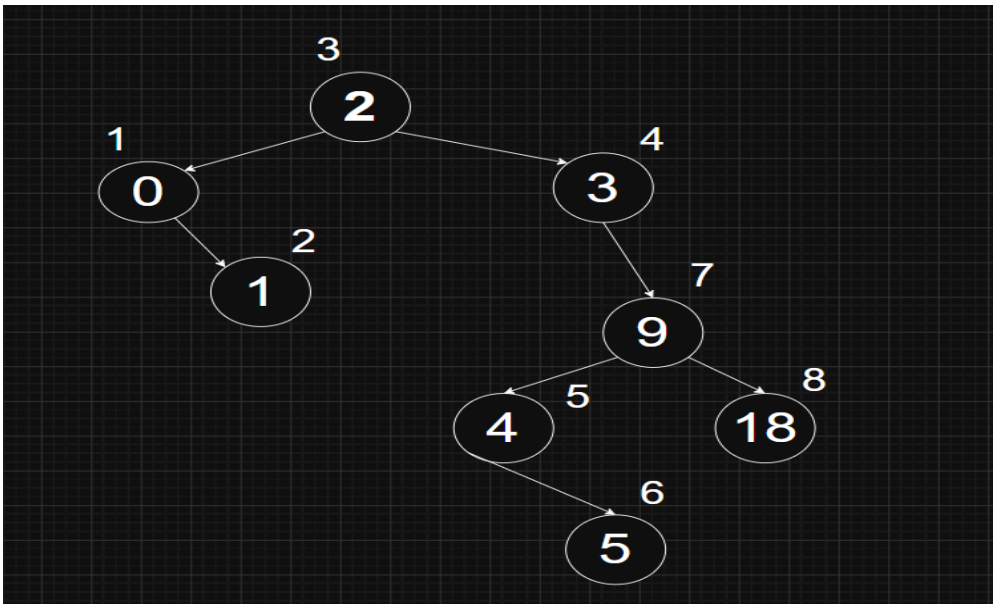
1. Creation of Pre-Order Traversal

Code + Console Screenshot + Tree Diagram

```
62  template<typename T>
63  void BinarySearchTree<T>::inOrder(){
64      if(left != nullptr){
65          left->inOrder();
66      }
67      std::cout<< key << "     ";
68      if(right != nullptr){
69          right->inOrder();
70      }
71  }
```

```
19      std::cout<<"In Order Output: ";
20      tree->inOrder();
21      std::cout<<std::endl;
```

```
In Order Output: 0    1    2    3    4    5    9    18
```



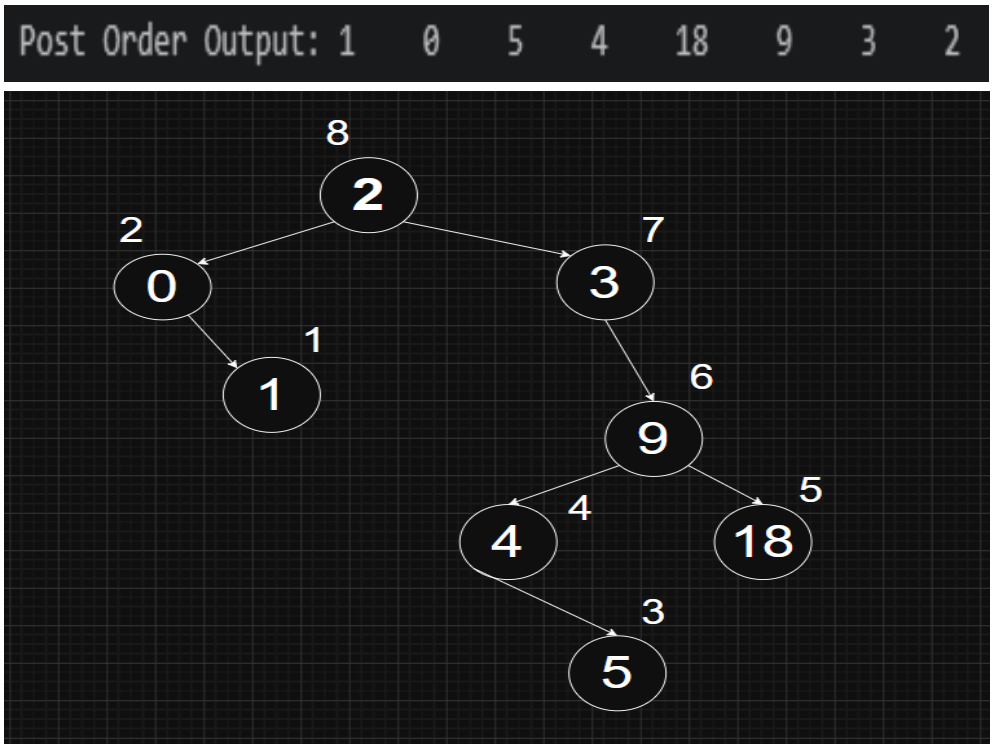
Observation	The in-order traversal was successfully implemented using the Binary Search Tree. The function first traverses the left subtree, displays the current node, and then traverses the right subtree. Since the values were inserted according to the Binary Search Tree property, the resulting output is arranged in ascending order. This demonstrates how in-order traversal can be used to display the stored values from smallest to largest.

2. Creation of In-Order Traversal

Code + Console Screenshot

```
73    template<typename T>
74    void BinarySearchTree<T>::postOrder(){
75        if(left != nullptr){
76            left->postOrder();
77        }
78        if(right != nullptr){
79            right->postOrder();
80        }
81        std::cout<< key << "     ";
82
83    }
```

```
25        std::cout<<"Post Order Output: ";
26        tree->postOrder();
27        std::cout<<std::endl;
```

	
Observation	The post-order traversal was successfully implemented using the Binary Search Tree. The function first traverses the left subtree, followed by the right subtree, and displays the current node last. The recursive process allows the child nodes to be visited before their parent node. The resulting output follows the left-right-root sequence required for post-order traversal.

3. Creation of Post-Order Traversal

CONCLUSION/LEARNINGS/ANALYSIS:
From this laboratory activity, it paved the way for me to understand how trees store data in a hierarchical manner, as well as how the structure of trees differs from linear data structures. Through programming General Trees and Binary Search Trees in C++, I was able to understand how the location of nodes affects their depth and height, while the traversal activities helped me understand the purposes of pre-order, in-order, and post-order traversals. Furthermore, I also observed how in-order traversal of a Binary Search Tree gives the values in ascending order, while pre-order and post-order follow different sequences depending on the visiting order of the root node. Other than that, I was able to extend my knowledge by understanding how traversal functions recursively work, where each subtree is visited before returning to its parent node. From the supplementary activity, which was already provided by my professor, I was able to reinforce my knowledge by working with the given BST implementation and its traversal functions. The only part that I needed to create was the tree diagram and the indicated traversal orders. Creating the diagram helped me understand the concepts of pre-order, in-order, and post-order traversal more clearly because I was able to visualize the actual order in which each node is visited and compare it with the outputs produced by the program.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: Grammarly and ChatGPT

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: Code Assistance & Verification / Grammar Checker

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: The AI only helped me debug my code and verified if I met the required HOA's instruction and criteria. Grammarly was also used in order to catch my grammatical errors. Other than that, the AI did not replace my idea and what I want to implement in my code.

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.